

2026-06-15 MON

Meetings: 1 h

- 9-9:30 w/ Kri on Dashboard IISS-530
- 9-30:10 Scrum

Lunch: 30 m

Work Log — IISS-487: Meridian SRE Dashboard Panel Documentation & Restructuring

1. Row Documentation — MSK & DocumentDB (Rows 7–8)

Documentation files were authored for the MSK (Row 7) and DocumentDB (Row 8) service rows, covering panel purpose, CloudWatch metric sources, statistical aggregation methods, and operational threshold guidance. Each file follows the established documentation format used across earlier rows, ensuring consistency for both current operators and future maintainers onboarding to the dashboard.

2. Row Documentation — RDS & Redis (Rows 9–10) & Inventory File Numbering

Documentation files were completed for the RDS (Row 9) and Redis (Row 10) service rows, bringing all previously undocumented rows up to the same standard. In addition, all row inventory files across the project were renamed with a numeric prefix matching their row position in the dashboard, making it straightforward to locate the correct documentation file without needing to cross-reference the dashboard structure.

3. Health Row to Performance Row — Click-Through Deep-Link Navigation

Data links were added to all stat panels in the Health Overview row (Row 1) so that clicking a health indicator navigates directly to the corresponding bargauge panel in the Performance row (Row 2). This creates a two-level drill-down path, allowing operators to scan the health row for anomalies and immediately access the relevant performance detail without manually scrolling through the dashboard. Links use panel ID anchors to ensure navigation lands precisely on the target panel.

4. Resource Table Panels — AWS Console Deep-Links for SQS, Lambda & ECS

Resource table panels with AWS Console deep-links were added to the SQS, Lambda, and ECS service rows. Each table displays the resource inventory for that service and provides a single-click link from each row to the corresponding resource page in the AWS Console, giving operators a direct path from the dashboard to live resource configuration and metrics during incident investigation.

5. Resource Table Panels — AWS Console Deep-Links for MSK, DocumentDB, RDS, Redis, Neptune & OpenSearch

Resource table panels with AWS Console deep-links were extended to the remaining service rows: MSK, DocumentDB, RDS, Redis, Neptune, and OpenSearch. Completing the deep-link rollout across all service rows ensures that operators have consistent one-click console navigation regardless of which service row they are investigating, eliminating the need to manually construct console URLs during time-sensitive incidents.

6. Resource Table Panels — renameByRegex CloudWatch Index Prefix Strip

A `renameByRegex` transformation was applied to nine resource table panels to address a CloudWatch behaviour where metric query results are prefixed with a numeric index (e.g., `"1 - queue-name"`). The transformation strips the prefix and displays clean resource names across all affected panels. The `renameByRegex` technique guide and relevant panel documentation were updated to document this as the standard pattern for all current and future resource table implementations.

7. SQS Row Restructure — Separate Standard Queue & DLQ Resource Tables

The SQS row was restructured to separate standard queues and dead-letter queues into two distinct resource table panels. Previously, both queue types were displayed together, making it difficult to distinguish between healthy queue activity and DLQ accumulation indicating processing failures. The dedicated DLQ table gives operators focused visibility into dead-letter queue depth independently from standard queue metrics.

8. SQS Row — Four-Panel Metric Row Addition

A four-panel metric row was added to the SQS row below the resource tables, with each panel set to width 6. This follows the layout convention established across other service rows and provides dedicated panels for key SQS operational metrics alongside the resource inventory

tables, completing the SQS row restructure into a consistent and comprehensive service monitoring view.

9. Health Row SQS Panel — DLQ Exclusion via `filterFieldsByName`

The SQS Health stat panel in the Health Overview row was including dead-letter queue metrics in its aggregation, inflating apparent queue depth and producing misleading health indicators. A `filterFieldsByName` exclude pattern was added to filter out DLQ queues from the health calculation, ensuring the stat panel reflects only standard queue depth. DLQ monitoring is now handled separately through the dedicated DLQ resource table in the SQS row.

10. Performance Row SQS Panel — DLQ Exclusion, Top-K Display & Description

The SQS bargauge in the Performance row was updated to exclude DLQ queues, consistent with the health row exclusion applied in the same session. The panel was configured to display only the top K queues by depth, focusing operator attention on the highest-volume queues rather than the full inventory. A panel description was added explaining the DLQ exclusion rationale and the top-K selection criteria for future reference.

11. SQS Resource Tables — `sortBy` Removal & Max Reducer for DLQ Table

Two issues were resolved in the SQS resource tables. The `sortBy` transformation was removed from both tables as it was producing display ordering that did not reflect operational priority. The DLQ table reducer was switched to `max`, ensuring the table surfaces the peak message depth observed within the selected time range rather than an average that could mask transient spikes. Both changes align the SQS tables with the resource table standard applied across all service rows.

12. Lambda Row — Error Logs Panel Removal & Reducer Standardisation

The Lambda error logs panel was removed following a decision to defer CloudWatch Logs integration to a future sprint, eliminating a no-data placeholder that would have caused operator confusion. The Lambda resource table reducer was simultaneously switched to `max` to align with the standardisation applied to resource tables across all other service rows during this session.

13. Lambda Row Restructure — Resource Table Height & Metric Panel Layout

The Lambda row layout was restructured to set the resource table height to 8, providing

sufficient vertical space to display multiple Lambda functions without truncation. Metric panels were reorganised into groups of four per visual row at width 6 each, bringing the Lambda row into full visual alignment with the layout convention established across SQS, ECS, and other service rows.

14. Resource Table Global Standardisation — Height, Reducer & Sort

A cross-row audit of all resource table panels identified inconsistencies in height settings, reducer configuration, and sort transformations. All resource tables across every service row were standardised to height 8, the `max` reducer, and removal of `sortBy` transformations. This ensures consistent behaviour and appearance throughout the dashboard and provides a single known-good configuration template for developers adding resource tables to future service rows.

15. ECS Infrastructure Row — 4-Panel Layout & Barchart Addition

The ECS Infrastructure row was restructured to a four-panels-per-row layout at width 6, consistent with the layout standard applied across other service rows. A barchart panel was added to provide a visual distribution of ECS task counts across services, giving operators an at-a-glance view of workload distribution across the ECS cluster alongside the existing infrastructure health metrics.

16. Unhealthy ALB Targets Panel — Relocation to ECS APIs Row

The Unhealthy ALB Targets panel was relocated from the ECS Infrastructure row to the ECS APIs row, where it logically belongs. ALB target health is directly related to API availability and request routing rather than infrastructure capacity, making the ECS APIs row the appropriate context for this metric. The relocation improves the logical coherence of both rows and ensures operators investigating API health issues find all relevant panels in one place.

17. ECS APIs Row — Metric Panel & Response Time Percentile Layout Reorder

The ECS APIs row was reordered to present four metric panels at width 6 followed by three response time percentile panels (p50, p95, p99) at width 8. The wider allocation for percentile panels provides additional horizontal space to distinguish between latency bands clearly. Placing throughput and error rate panels first ensures the most frequently consulted operational metrics are immediately visible before the latency detail section.

18. Unhealthy ALB Targets Panel — Legend Cleanup via renameByRegex

Following the relocation of the Unhealthy ALB Targets panel to the ECS APIs row, the panel legend was found to be displaying raw target group dimension values. A `renameByRegex` transformation was applied to extract and display clean target group names, consistent with the legend standardisation applied to all other ECS APIs panels. This ensures the panel integrates visually and functionally with its new row context.